

Ayudantía 9: Repaso árboles y grafos

Profesores: Sebastián Sáez, Diego Ramos

Ayudantes: Diego Duhalde, Benjamín Wiedmaier, Fernando Zamora

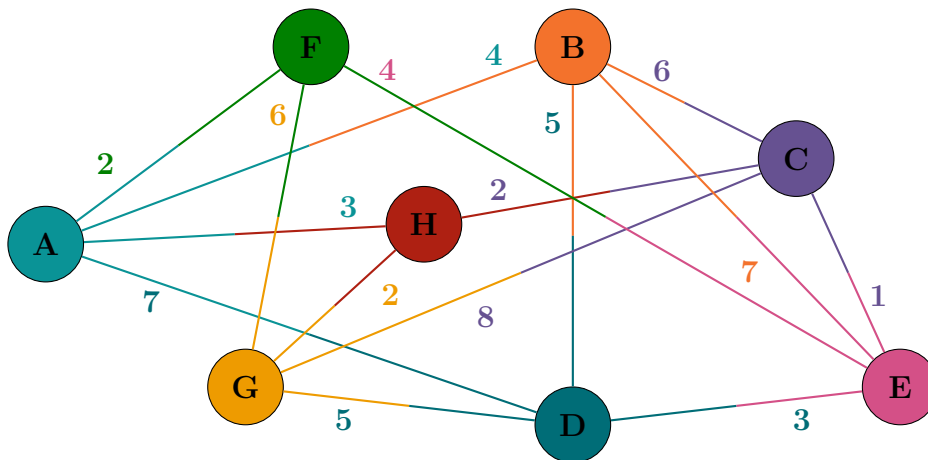
PREGUNTAS

1. Defina qué es un árbol AVL y qué propiedades de balance lo caracterizan.
2. Escriba en C una función que permita **insertar nodos** en un árbol auto-balanceado, considere incluir todas las operaciones necesarias.
3. Dado un árbol AVL inicialmente vacío, inserte en orden las claves

50, 30, 70, 20, 40, 60, 80, 10, 5

y para cada paso:

- a) Dibuje la estructura del árbol señalando alturas de cada nodo.
 - b) Indique si tras la inserción se realizó alguna rotación; en caso afirmativo, especifique cuál.
4. Considere el siguiente grafo G:



- a) Aplique el algoritmo de Kruskal para encontrar el árbol de expansión mínima del grafo. Indique el orden en que se agregan las aristas y dibuje el árbol resultante.
- b) Aplique el algoritmo de Dijkstra para encontrar el camino más corto desde el nodo A a todos los demás nodos. Indique el orden en que se visitan los nodos y las distancias mínimas encontradas. Muestre la tabla de distancias finales para cada nodo.

RESPUESTAS

1. Un árbol AVL es un tipo de árbol binario de búsqueda auto-balanceado en el que, para cada nodo, la diferencia de alturas entre sus subárboles izquierdo y derecho (llamada factor de balance) es a lo sumo 1, es decir:

$$|\text{altura}(\text{subárbol izquierdo}) - \text{altura}(\text{subárbol derecho})| \leq 1$$

Esta propiedad garantiza que las operaciones de inserción, eliminación y búsqueda tengan una complejidad temporal $O(\log n)$, manteniendo el árbol balanceado mediante rotaciones cuando es necesario.

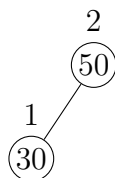
2. La implementación del código la puede encontrar en la rama de ayudantías del repositorio del curso. [Link](#).
3. a) Paso a paso de las inserciones y alturas:

- **Insertar 50:**



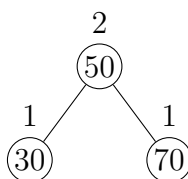
Altura de 50: 1. No hay rotación.

- **Insertar 30:**



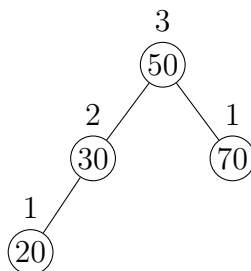
Alturas: 50 (2), 30 (1). No hay rotación.

- **Insertar 70:**



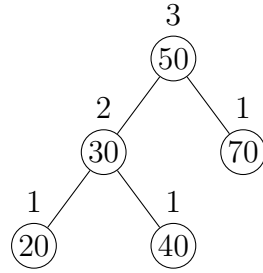
Alturas: 50 (2), 30 (1), 70 (1). No hay rotación.

- **Insertar 20:**



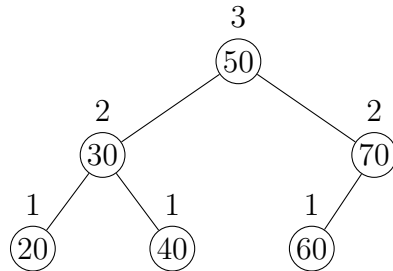
Alturas: 50 (3), 30 (2), 20 (1), 70 (1). No hay rotación.

- Insertar 40:



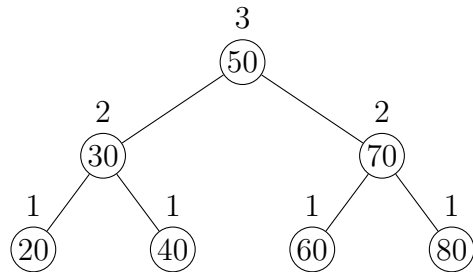
Alturas: 50 (3), 30 (2), 20 (1), 40 (1), 70 (1). No hay rotación.

- Insertar 60:



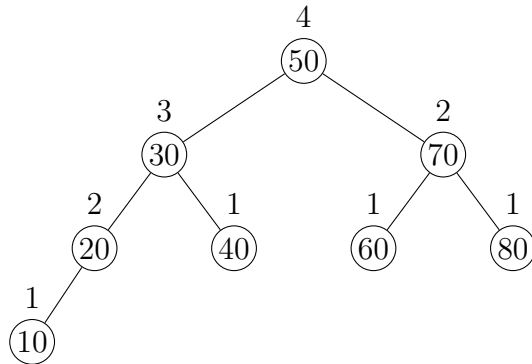
Alturas: 50 (3), 30 (2), 70 (2), 20 (1), 40 (1), 60 (1). No hay rotación.

- Insertar 80:



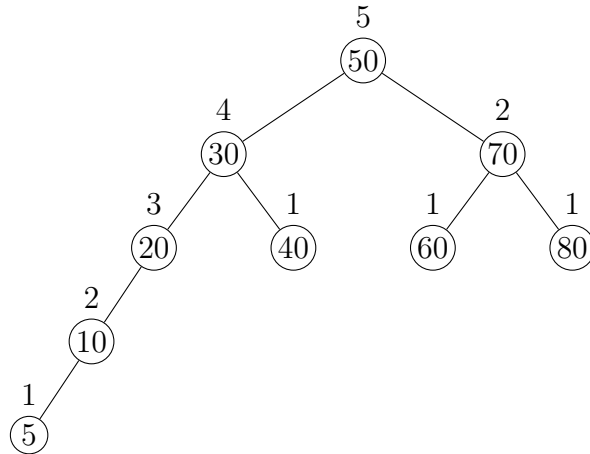
Alturas: 50 (3), 30 (2), 70 (2), 20 (1), 40 (1), 60 (1), 80 (1). No hay rotación.

- Insertar 10:



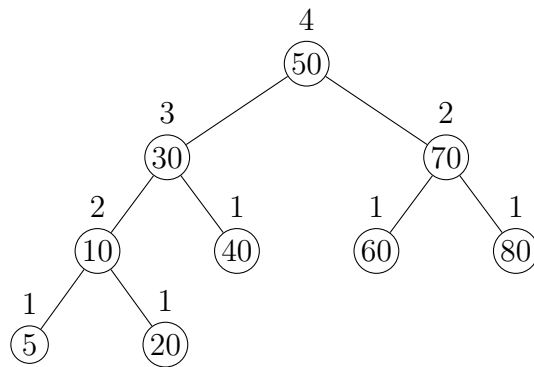
Alturas: 50 (4), 30 (3), 70 (2), 20 (2), 40 (1), 60 (1), 80 (1), 10 (1). No hay rotación.

- **Insertar 5:**



Alturas: 50 (5), 30 (4), 20 (3), 70 (2), 10 (2), 5 (1), 40 (1), 60 (1), 80 (1).

Rotación: Hay un desbalance en el nodo 30 (factor de balance = 3 en el subárbol izquierdo y 1 en el derecho), por lo que se realiza una **rotación simple a la derecha** en el nodo 20 para balancear el árbol. El árbol resultante es:



Alturas: 50 (4), 30 (3), 10 (2), 70 (2), 5 (1), 20 (1), 40 (1), 60 (1), 80 (1).

b) **Rotaciones realizadas:**

- Insertar 50: Ninguna rotación.
- Insertar 30: Ninguna rotación.
- Insertar 70: Ninguna rotación.
- Insertar 20: Ninguna rotación.
- Insertar 40: Ninguna rotación.
- Insertar 60: Ninguna rotación.
- Insertar 80: Ninguna rotación.
- Insertar 10: Ninguna rotación.
- Insertar 5: Rotación simple a la derecha con pivote en el nodo 20.

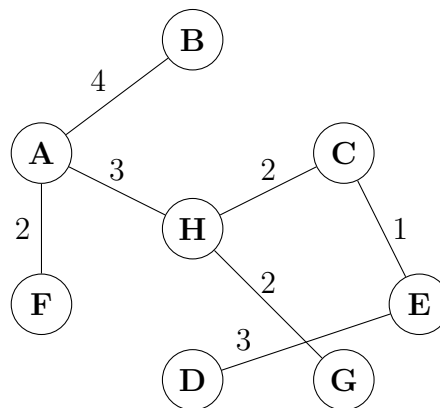
4. a) **Kruskal:** Ordenamos las todas las aristas de menor a mayor peso.

Arista	Peso
C-E	1
A-F	2
C-H	2
G-H	2
A-H	3
D-E	3
A-B	4
E-F	4
B-D	5
G-D	5
G-F	6
B-C	6
B-E	7
A-D	7
C-G	8

Seleccionamos las 7 aristas de menor peso sin formar ciclos:

- C-E (1)
- A-F (2)
- C-H (2)
- G-H (2)
- A-H (3)
- D-E (3)
- A-B (4)

Árbol resultante:



- b) **Dijkstra (desde A):**

Iniciamos las distancias $d(A) = 0$, $d(\text{resto}) = \infty$. Conjunto U vacío.

1. Tomamos **A** (dist=0). Actualizamos:

$$d(B) = 4, d(D) = 7, d(F) = 2, d(H) = 3.$$

2. Tomamos el menor en $V \notin U$: **F** (2). Actualizamos: $d(E) = 2 + 4 = 6$, $d(G) = 2 + 6 = 8$.
3. Siguiente: **H** (3). Actualizamos: $d(C) = 3 + 2 = 5$, $d(G) = \min(8, 3 + 2 = 5) = 5$.
4. Siguiente: **B** (4). Sus aristas no mejoran distancias.
5. Siguiente: **C** (5). Actualizamos: $d(E) = \min(6, 5 + 1 = 6) = 6$.
6. Siguiente: **G** (5). Actualiza $d(D) = \min(7, 5 + 5 = 10) = 7$.
7. Siguiente: **E** (6). Sus aristas no mejoran distancias.
8. **D** (7). Fin.

Orden de visita: A, F, H, B, C, E, G, D

Tabla de distancias finales:

Nodo	Distancia desde A
A	0
B	4
C	5
D	7
E	6
F	2
G	5
H	3